MACHINE LEARNING PROJECT

Julian LOPEZ

Fabrice ROSSI

Universite Paris dauphine
2025-2026

The objective of this report is to do an analysis and train a classification model using a random forest and a gradient boosting approach. The idea is to compare both models and then use the best one to predict and classify several non-seen observations. For this specific project, all the datasets were provided, no external data was used, and we do not know what the classification of the individuals meant. For this exercise the 2 classifications are I or T. The data that will be used show the socio-economic level of the people

Overall, the process that we will follow to conduct this project is the following:

1. Analysis of the raw main dataset
2. Create a complete dataset
3. Data analysis of new variables
4. Data cleaning
5. Training of the 1st model: Random Forest
6. Training of the 2nd model: Gradient boosting
7. Compare both trained models
8. Predictions

1. DATA ANALYSIS

In the dataset provided, we have several csv files that we will use to train the models. Among all these files, there are 2 main files that we will first focus on ***learn_dataset*** and ***test_dataset***. We will first do an analysis of the variables in these 2 files, especially on our objective variable Y (which is present only on the ***learn_dataset***, column target).

When checking the quality of the data on these 2 files, we see that there are no missing or duplicated identifiers nor missing values. About the data types, they are as follows:

- 1 numeric variable
- 8 categorical variables
- 1 boolean variable

Learn dataset

Shape: (50046, 10)
Duplicate rows: 0

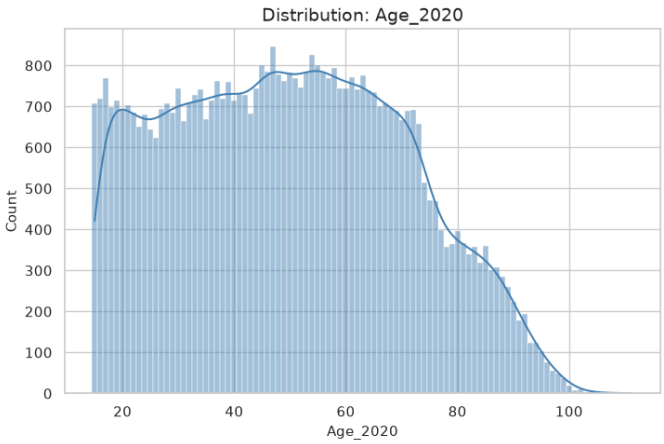
	dtype	non_null	missing	missing_%	unique
Pkey	str	50046	0	0.0	50046
INSEE_CODE	str	50046	0	0.0	13514
highest_credential	str	50046	0	0.0	12
Job_42	str	50046	0	0.0	41
Age_2020	int64	50046	0	0.0	95
HOUSEHOLD	str	50046	0	0.0	9
Sex	str	50046	0	0.0	2
Act	str	50046	0	0.0	6
IS_STUDENT	bool	50046	0	0.0	2
target	str	50046	0	0.0	2

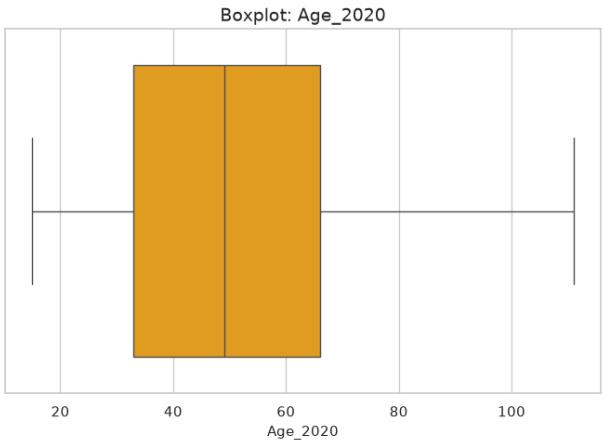
```
--- Learn persons ---
Rows: 50046
Unique IDs: 50046
Missing IDs: 0
Duplicated IDs: 0
--- Test persons ---
Rows: 50045
Unique IDs: 50045
Missing IDs: 0
Duplicated IDs: 0
```

Analysis of the numeric variables

The only numeric variable available is ***age_2020***. This variable has no missing values, and we can see that it has an expected distribution where there is a concentration of data on the ages 20 to 60 and a fast decline in the amount of data on the age 70 and onwards. Thanks to the boxplot below we can identify a longer right tail, indicating that there are very high values present in this variable. This is, however, normal and expected as well since we

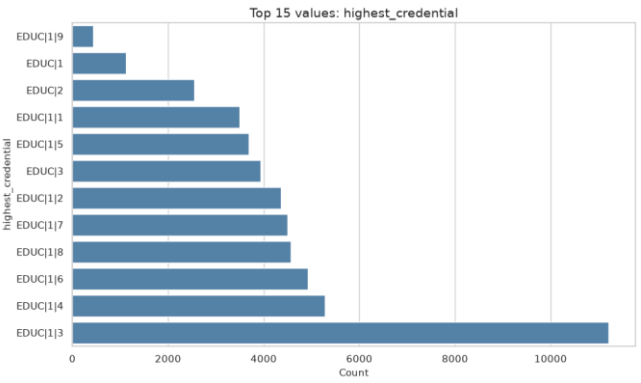
expect to have people reaching 90+ years old but being very few of them. We can then conclude that there are no outliers to worry about and the values and distribution of the variable behaves as expected

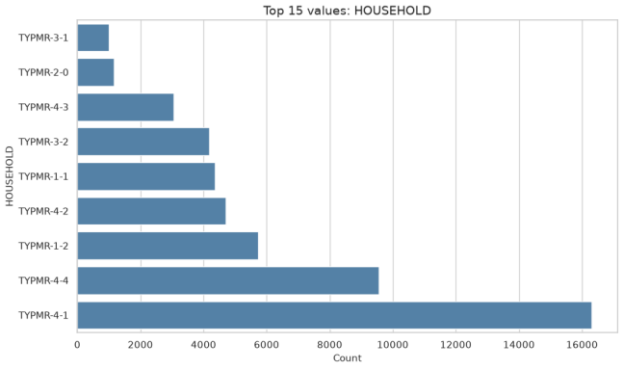


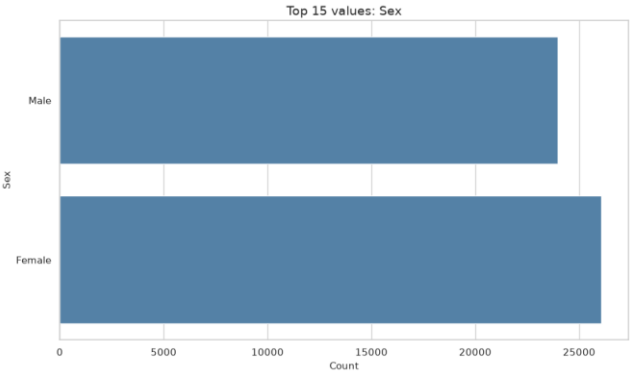


Analysis of the categorical variables

For the categorical variables, there is a clear unbalance in their categories for variables ***highest_credential***, ***household***, ***act*** and ***is_student***. On the other hand, variables ***sex*** and ***target*** have a more equitable distribution where we have a similar number of observations for all categories





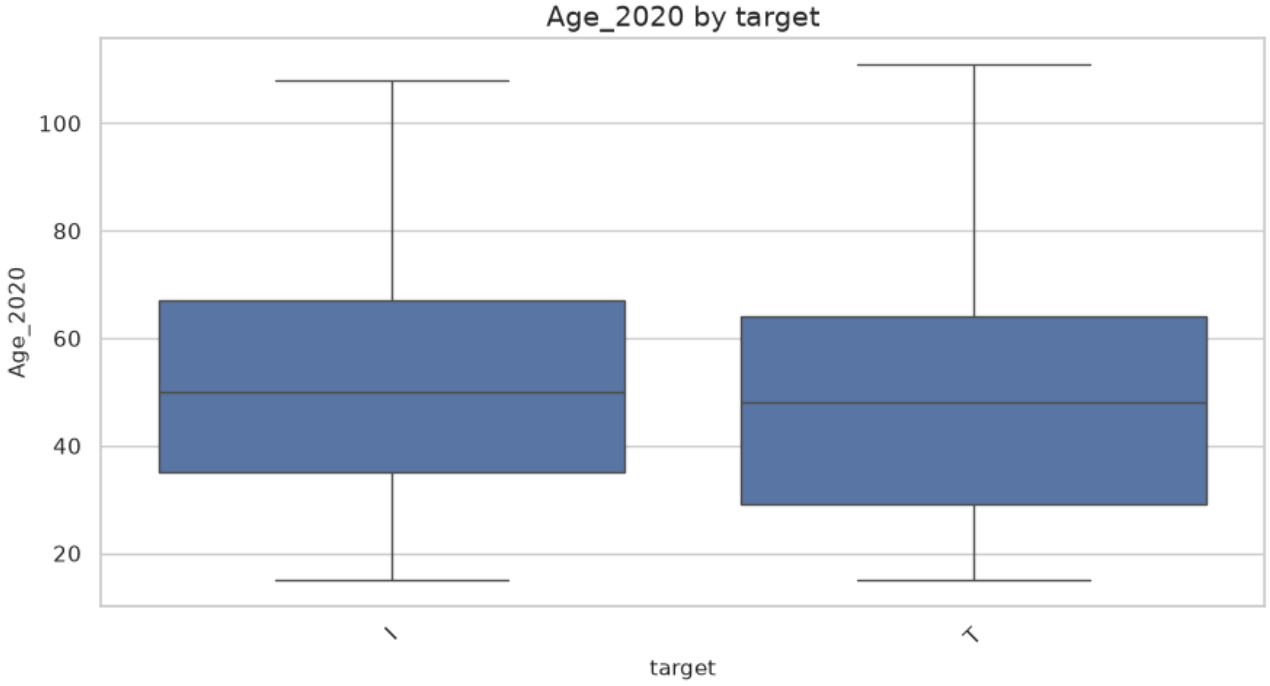


Analysis of the variable *target*

We can see that variable ***target*** has almost a 50-50 distribution between both categories, as we have 27702 observations for category I and 22344 observations for category T (55.35% and 44.65% of the data respectively)

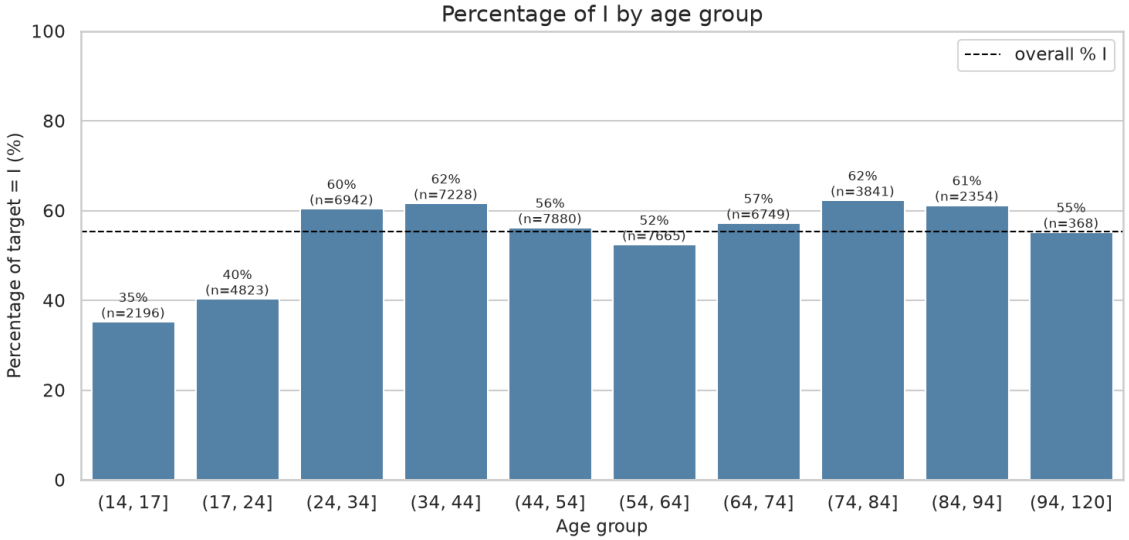


We can then compare our numeric variable ***age_2020*** against our variable ***target*** to extract more information from it. We can see that the age does not change much with the target category given. It is seen that the average for both groups is similar and overall, they both have a similar global distribution.



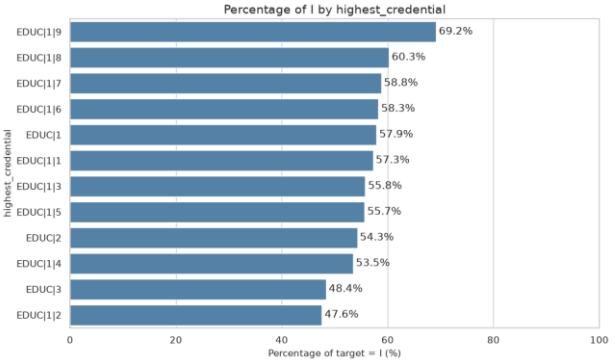
If we group the people by age group and compare both groups however we can see that there is a higher percentage of people who belong to category T vs category I in the group

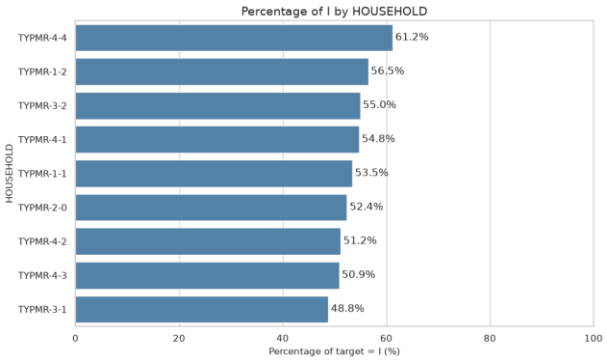
under 24 years. It is mostly balanced for the rest of the groups. With this we can conclude that the age proportion will make the proportion of category I and T vary.

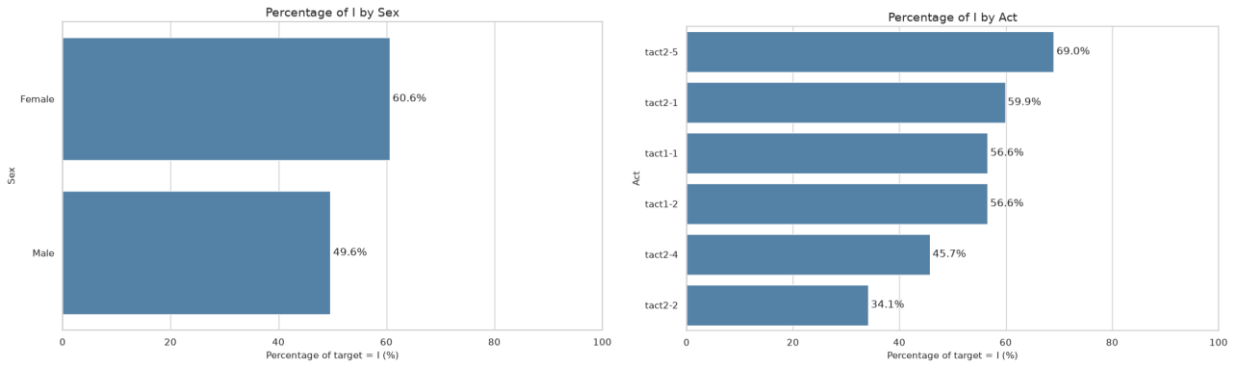


If we now compare our categorical variables against the variable ***target,*** we can see that:

- ***Highest_credential***: Equitable distribution for both I and T categories. Only exception is category EDUC 1-9 that has a higher concentration on the I group (69%) vs the T group (31%)
- ***Household***: Equitable distribution. Almost 50-50 between both groups
- ***Sex***: Asymmetric distribution. It is seen that 60% of women belong to group I vs 40% on group T. For men the distribution is 50-50
- ***Act***: Asymmetric distribution. Some categories are more present on group I (like tact 2-5 or tact 2-1 with a 69% and 60% respectively) vs other categories being more present on group T (like tact 2-2 with a 66%)
- ***Is_student***: Asymmetric distribution. People that are students mostly belong to group T (68% of students are in this group vs 32% belonging to group I) while people that is not a student are mostly on group I (59% of not students are in this group vs 41% of non students belonging to group T)



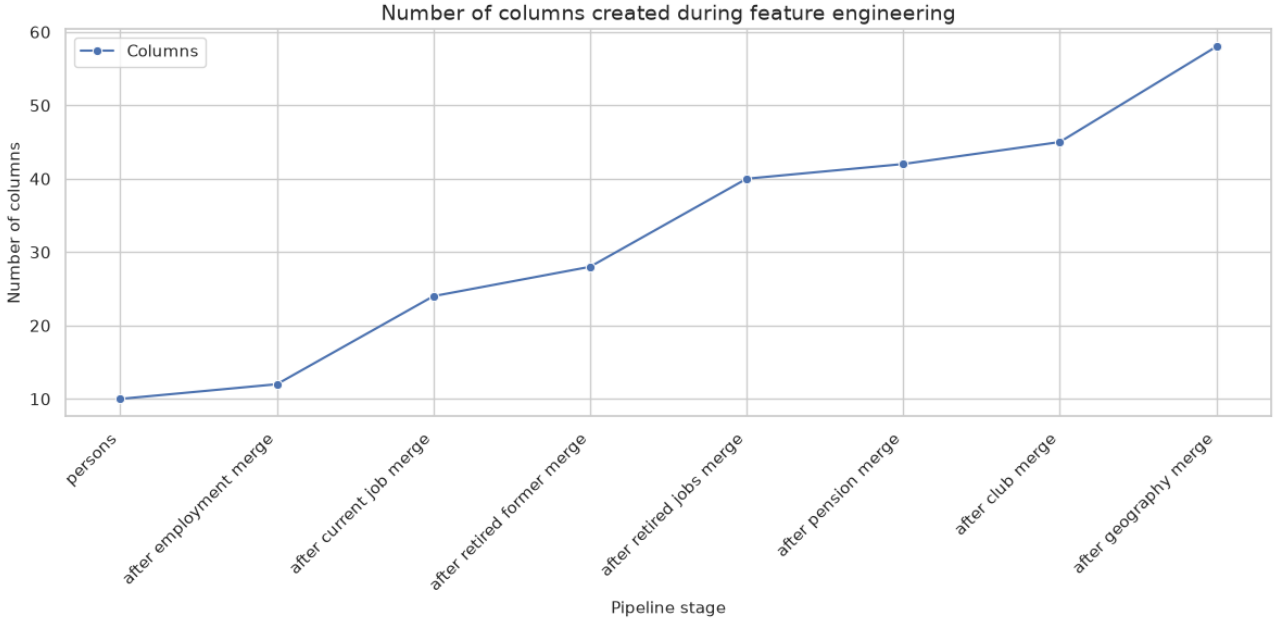




2. CREATE A COMPLETE DATASET

As stated in the beginning, we are provided with several files containing extra information that we can use to enrich our database. In this step we will be combining all the available files and creating a comprehensive dataset to be used to train the models.

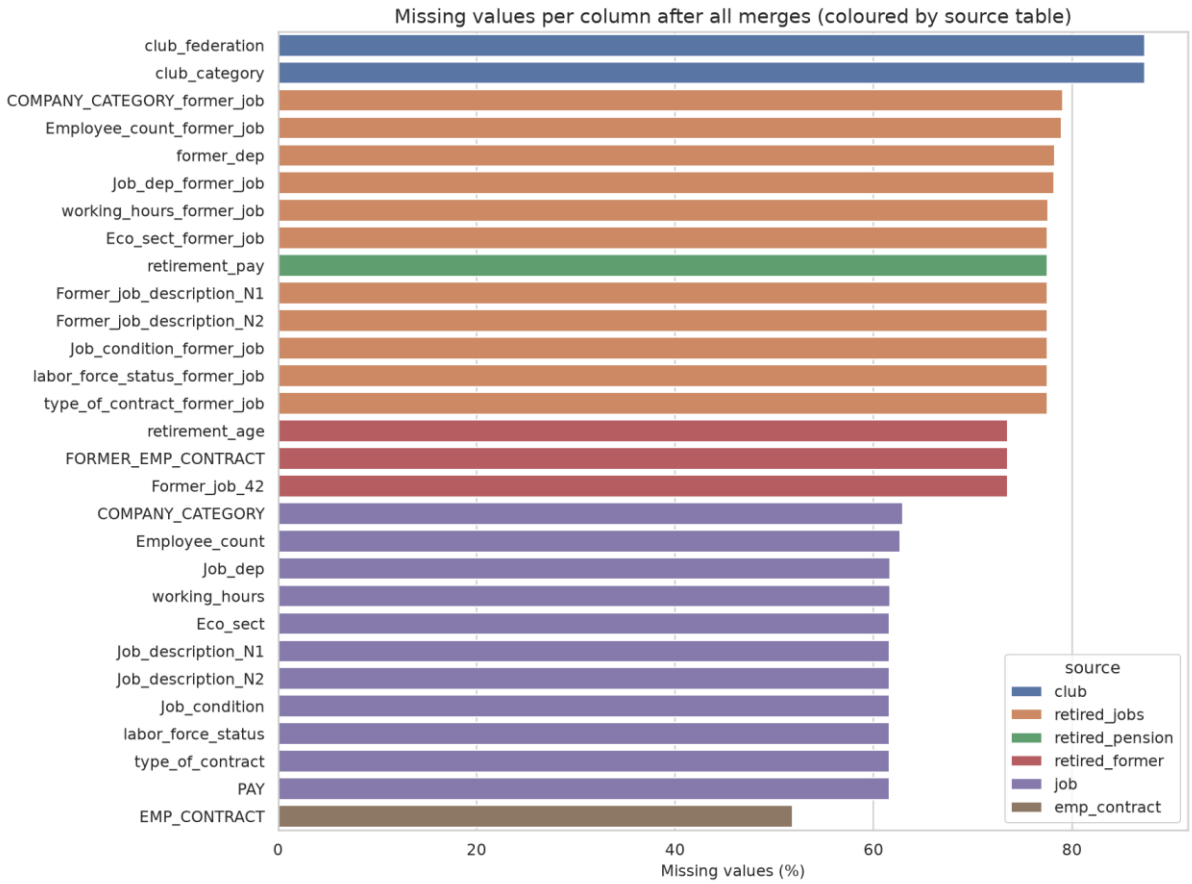
To do this, we will do a join on all tables using column ***Pkey*** as the primary key. After this is done, we see that we get a final dataset containing more than 50 variables (vs the original 9 variables on the main dataset analyzed previously)



When creating this comprehensive dataset, a lot of missing values are added since there is missing information on the extra tables joined. Given the nature of the data (socio economic data based on job/retirement status and conditions) we need to ask ourselves the following question

"Is an empty cell really a missing value?"

We could have the case of an unemployed person who, by definition, has no job or salary, so the auxiliary tables will, by definition, be empty for this person. For cases like this, the missing data is not really a missing value, so we need to differentiate the real missing values from the expected missing values (which we will call structural missing values).



To do so, we will use table ***EMP_Contract***, where we have a list of all employed people. Then with tables ***job***, ***retired_former***, ***retired_jobs*** and ***retired_person*** we will get all retired people. Combining these two we will get all working and retired people (so, all people are concerned in our study). When checking this we got a total of 1,028,540 empty cells. If we compare it with the 1,031,951 total empty cells we get there is 99.7% of empty cells belonging to the structural missing values (expected missing values). Since these “missing” data is correct, we will keep it in the database as the empty cells are also adding information useful for the model.

On the other hand, we have the real missing values, which is just 0.3% of the empty cells on the complete dataset. This is a very low percentage of missing values, and we could remove the observations having real missing values, however we wont since this is also useful to train the model. Since the distribution of the data in both test and train tables is the same, these missing data are also informative because the model can learn on them and find the patterns belonging to the missing data on the test dataset (data never seen before) and help it classify correctly the people with missing data.

Finally, we can do a recap of the variables that we have in the comprehensive dataset:

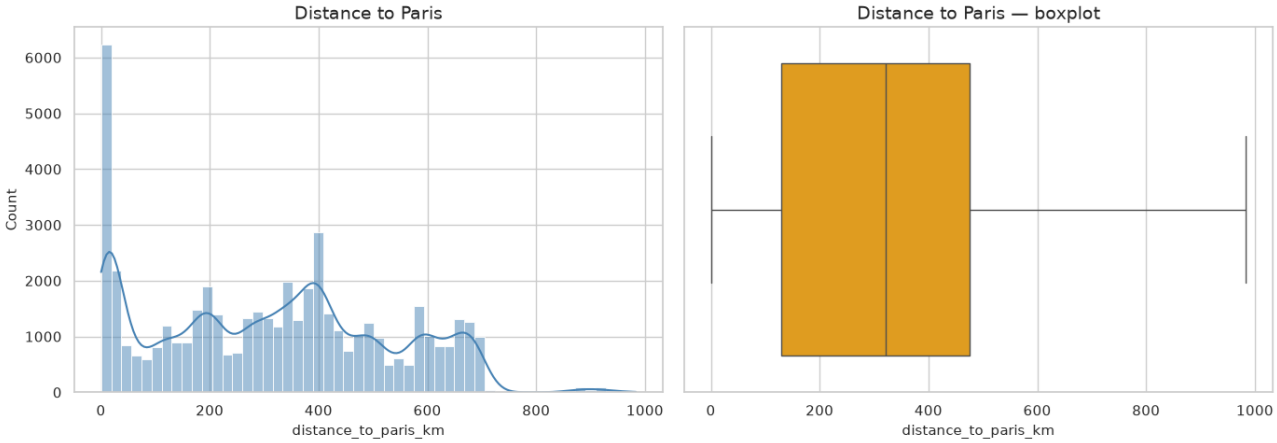
- 14 numeric variables
- 7 boolean variables
- 37 categorical variables

3. ANALYSIS OF THE NEW VARIABLES

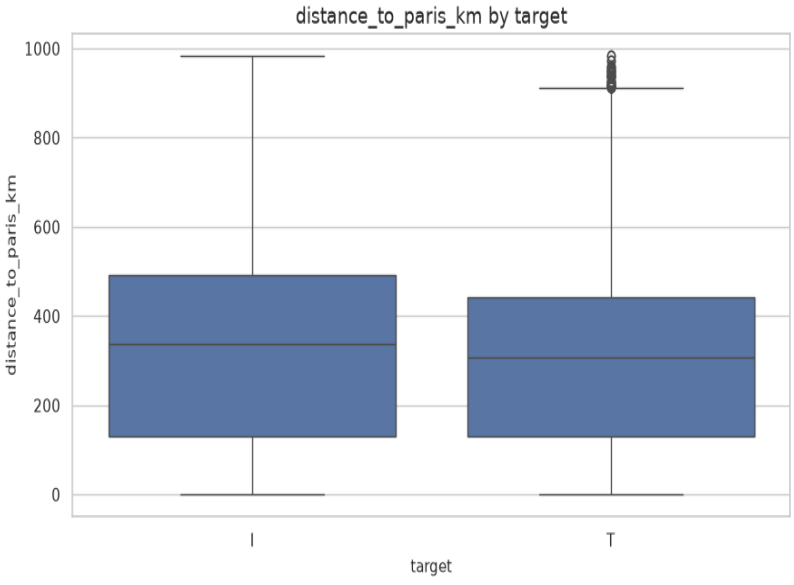
Since several new variables were added, is important to check them as well and see if there is important or problematic information that we will have to consider before training the model. Seeing that more than 40 variables were added, we won't do an analysis on all of them, so we will present only the most important ones next.

Distance variables

As part of the new comprehensive dataset, some geographical variables were added. Among these, some of them were created with a purpose, for example variable ***distance_to_paris_km***. This variable was created to identify the real distance to Paris and help the model by giving it some useful extra information. If we check its distribution, we see that we have a peak on 0 which is the people living in Paris. We can also see a it follows a flat and wide distribution with a median on 322km. Checking the boxplot we can also see a longer right tail, showing there is a low number of people living more than 800km away from Paris

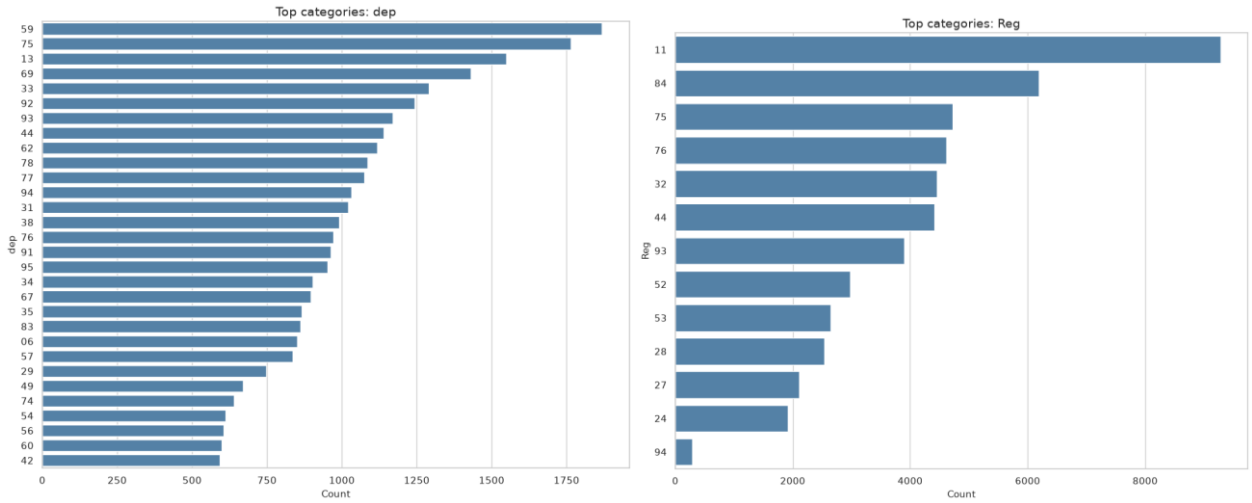


If we check now how this variable behaves against the variable ***target***, we see that the mean of the distribution of distance to Paris for group I is a little higher than the mean of group T. This shows that individuals belonging to group T are slightly closer to Paris. We can also see that there are some observations shown as individual points rather than belonging to the tail of the boxplot, meaning that there are less cases of people living further than 800km from Paris for group T vs group I



We can also check the distribution of people per department and region. We can see that data has a similar frequency for the departments, where all the categories have a similar

amount of data. For the regions, however, we see a difference in the frequencies as there is a high concentration of data on region 11 and very few on region 94. This is expected as region 11 (ile de France) is the most populated region and region 94 (Corse) is the least populated region.



4. DATA CLEANING

The data cleaning of the dataset for the project was made in 2 different steps. The first one was made when all the tables were joined together. A first data cleaning process was to define the variables in their correct datatype (***Pkey*** or ***INSEE_CODE*** for instance were loaded as text instead of numeric)

Quality control was also considered in this step. As previously mentioned, it was verified that there were no duplicated lines nor missing identifiers when joining the tables. We also ensured to maintain all the observations for both the train and test dataset (50,047 and 50,046). And finally with the variable analysis we identified there were no outliers to remove from the dataset.

Moreover, a reduction in the categories was made whenever possible. In the case of ***job_description*** for example, around 400 values belonging to N3 were replaced by N2 and N1, which contain around 29 values on both of them combined, which will help the model classifying on the training step later.

Some variables were also added to help the model. As said before, variable ***distance_to_paris*** is an example of this or all variables starting with "***has_"*** where added as well to be used as a flag. On this matter, variable ***log_inhabitants*** was also added to correct the asymmetry on the population distribution.

Other than this, some variables are not useful to train the model were removed so that the model will keep only the variables with a real impact on determining the classification of the people. Variables such as ***Pkey*** (unnecessary id) or ***INSEE_CODE*** (high number of categories not helpful to classify) are an example of this. We also removed redundant variables like ***department_name*** and ***region_name*** among others.

The second step of the cleaning was made directly on the pipeline of each model training. For the random forest model, we did the following:

- Structural missing values of categorical variables are replaced with a new category “missing”. Then a one hot encoding step is applied
- Structural missing values of numeric variables and Boolean variables are replaced with the median of each variable

For the gradient boosting model, the approach is a little different. Since we use the package **HistGradientBoostingClassifier**, this package has included internally instructions to handle the missing data. For the structural missing values in this case no extra step was taken since the package knows how to handle these situations.

5. 1ST MODEL: RANDOM FOREST

Once the data is cleaned and the dataset is ready, we start creating and training our models. The first model that we will be creating is a random forest one. For this we will use package ***sklearn*** for both data cleaning and model training. On top of this, we will use a grid of the hyperparameters to test multiple models and choose the best one among them.

The process that will be followed is:
- Data is processed in the pipeline (missing values replaced)
- Set up a hyperparameter grid
- Set up a cross-validation strategy using a stratified Kfold (using 5 folds)
- Load each hyperparameter combination
- Train the model with each hyperparameter combination and save the result

We will repeat this process until we have trained a model with all possible hyperparameter combinations. The definition of the hyperparameters comes from a grid with the following hyperparameters:
- ***N_estimators*** (300): Quantity of decision trees in the forest. More trees help to reduce the variance of the prediction without overfitting.
- ***Max_depth*** (none, 20, 30): depth of each tree. Useful to control tree complexity
- ***Min_samples_leaf*** (1, 5, 20): Minimum observations required per leaf. Useful to set rules based on observations (like allowing a leaf with one observation)
- ***Max_features*** (sqrt, 0.3): Proportion of features considered when searching for the best possible split

The idea behind using this hyperparameter grid is to test several types of models and get the best one. We want to try very flexible models (no depth limit and having 1 observation leaves for example), very regularized models (limited depth and controlling leaf size) and everything in between to get the best possible model.

Following this grid and its options, we have 18 possible combinations of hyperparameters. Since we are using 5 folds to train and test each model, we will get a total of 90 cross validation model fits, from which we will get the best one using the criteria ROC AUC. We want to get the highest possible ROC AUC.

After training the models and getting the best result, we see that the best model is a random forest with some level of regularization. For the best model we got a ***ROC AUC*** of 0.794 and a hyperparameter grid of
- ***max_depth***: 20
- ***max_features***': 0.3
- ***min_samples_leaf***: 5

- ***n_estimators***: 300

6. 2nd MODEL: Gradient boosting using *optuna*

The second model we will be using is a gradient boosting using packages **HistGradientBoostingClassifier** and ***optuna***. The process that we will follow is very similar to the one used on the random forest model. Here we will also use a 5-fold cross validation, and we will also test several hyperparameters to test several models and find the best one. The hyperparameters that will be used are:

- ***learning_rate***: Determines how fast the model learns and how good it is to generalize
- ***max_iter***: Number of iterations (trees)
- ***max_leaf_nodes***: Determines maximum number of leaves per tree
- ***max_depth***: depth of each tree
- ***min_sample_leaf***: Minimum observations required per leaf
- ***l2_regularization***: Used to avoid overfitting by penalizing large leaves

In this case we are testing 40 different combinations of hyperparameters proposed directly by ***optuna***, and since we are using the 5 folds cross validation, we are training a total of 200 models. Finally, same as before, we are using the ***ROC AUC*** criteria to choose the best model.

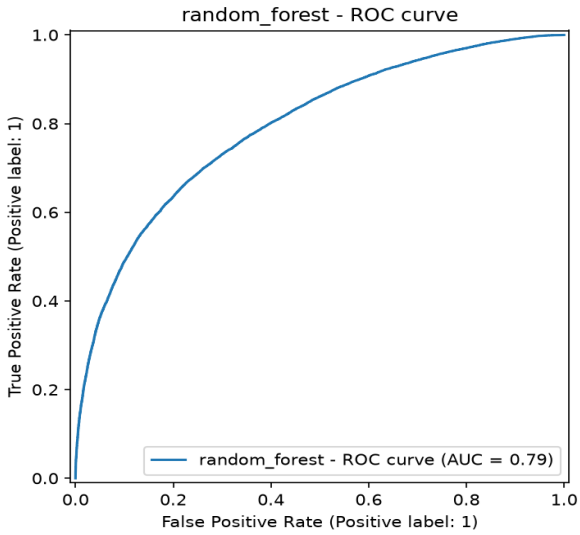
After training the models and getting the best result, we see that the best model is a model with a ROC AUC of 0.790 and a hyperparameter grid of

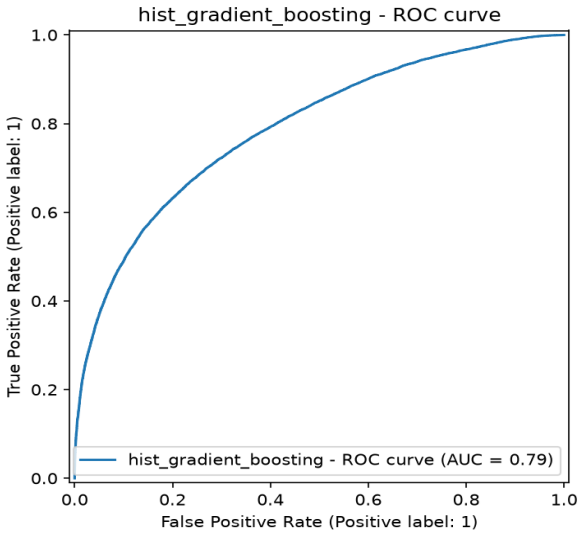
- ***learning_rate***: 0.02121154561370454
- ***max_iter***: 412
- ***max_leaf_nodes***: 20
- ***max_depth***: 12
- ***min_samples_leaf***: 20
- ***l2_regularization***: 0.281275665930269

This tells us that the best model is a model with some regularization, that learns slowly and does several iterations.

7. MODEL COMPARISON

Now that we have finished training the model and we have got the best performing model for both random forest and gradient boosting, we can now decide which one we want to use to make the prediction on the test dataset. As was presented previously, both models have a very similar ***ROC AUC*** value and curve, so we will use some extra criteria to identify the best model and choose one among the two. First, we can check the ***ROC AUC*** curves for both models

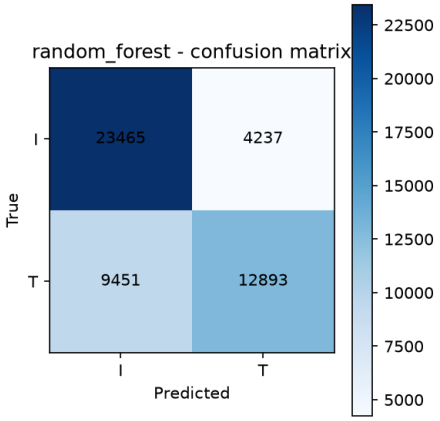




We can then use the confusion matrix to get the accuracy of the models, and this can give us more insights on which model to choose. The confusion matrix is as follows:

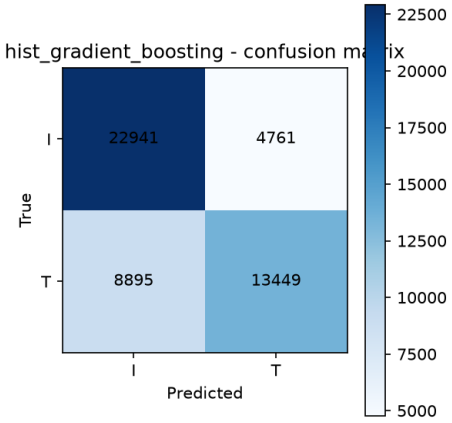
Random forest:

- Correct predictions
  - Predicted I that are I = 46.88%
  - Predicted T that are T = 25.76%
  -
- Errors
  - Predicted I that are T = 18.88%
  - Predicted T that are I = 8.46%



Gradient boosting:

- Correct predictions
  - Predicted I that are I = 45.84%
  - Predicted T that are T = 26.87%
- Errors
  - Predicted I that are T = 17.77%
  - Predicted T that are I = 9.51%



We get a total accuracy of 72.65% for random forest model and 72.71% for gradient boosting. This indicator is also close between both models (difference of 0.1%) and at first this

might not seem useful, but this is not the case. The question here and a big part of the decision will be based on the question

“What do I want to do/expect with my model?”

Based on these criteria we can tend to choose one model above the other, even if their total accuracy is practically the same. The difference comes from the fact that we can choose based on punishing more the false positives or the false negatives on the confusion matrix. For this specific case, however, this criterion is not totally useful, since we don’t know what the I and T groups are, so we don’t know which error type to punish more.

Since we cannot determine which error to minimize, we will set a main rule to select the model based solely on the highest ROC AUC, but seeing the results, both will behave very similarly.

8. Predictions

The random forest model will then be used to predict the classification of the test dataset. It will classify in groups I and T several observations not seen before by the model. Since both datasets train and test have almost the same distribution, we can expect a very similar performance on the test dataset. Moreover, besides the class, the model gives each person a probability of belonging to group T, and we can measure the quality of these probabilities with the Brier score: the mean squared error between the predicted probability and the real outcome (1 if the person is T, 0 if I). It was computed with the same 5-fold cross-validation, so every probability comes from a model that never saw that person. Finally, we can determine that the model will improve the classification by around 26%, passing from an accuracy of 55% (based on the distribution of I and T in the train dataset) to an accuracy of around 72-73%